Data Structures

CSL202

Quiz Set-A

1. Design and implement a **queue** using a **linked list**. The queue should support the following operations(in constant time):

   enqueue(x):

   dequeue():

   peek():

   isEmpty():

```
class QueueNode
{
 public:
 int val;
 QueueNode *next;
 QueueNode(int data)
  {val = data;
  next = nullptr;
  }};
```

   Input: enqueue(10),enqueue(20),

   dequeue(),enqueue(40),dequeue()

   Output: 10,20

2. Design and implement a **queue** using **two stacks**. The queue should support the following operations:

   enqueue(x),dequeue(),peek(),isEmpty().

   Input: enqueue(10),enqueue(20),dequeue(),enqueue(40),dequeue()

   Output: 10,20

Data Structures

CSL202

Quiz Set-B

1. Design and implement a **queue** using a **circular array**. The queue should support the following operations:

   enqueue(x),dequeue(),peek(),isEmpty().

   Input: enqueue(10),enqueue(20),dequeue(),enqueue(40),dequeue()

   Output: 10,20

2. Design and implement a **queue** using **two stacks**. The queue should support the following operations:

   enqueue(x),dequeue(),peek(),isEmpty().

   Input: enqueue(10),enqueue(20),dequeue(),enqueue(40),dequeue()

   Output: 10,20

3. The school cafeteria has **two types of sandwiches**:(0 → circular, 1 → square) Each student in a **queue** prefers either 0 or 1. Sandwiches are placed in a **stack** (index 0 = top).At each step:

- If the student at the front of the queue likes the sandwich on top of the stack, they take it and both leave.
- Otherwise, the student goes to the end of the queue.

This repeats until no student can take the top sandwich.Given:

- students[] = preferences of students in order (front to back of queue)
- sandwiches[] = sandwiches in stack order (top to bottom)

3. Given an integer k and a queue of integers, The task is to reverse the order of the first k elements of the queue, leaving the other elements in the same relative order.

   Only following standard operations are allowed on the queue.

   enqueue(x), dequeue(),size(),front().

   Input: q = 1 2 3 4 5, k = 3

   Output: 3 2 1 4 5

   Function : queue<int> reverseFirstK(queue<int> q, int k)

   {your code}

Return the **number of students who cannot eat**.

Input: students = [1,1,0,0], sandwiches = [0,1,0,1]

Output: 0

Function : int countStudents(vector<int>& students, vector<int>& sandwiches)

{your code}

Data Structures

CSL202

Quiz Set-C

1. Design and implement a **queue** using a **linked list**. The queue should support the following operations(in constant time):

   enqueue(x):

   dequeue():

   peek():

   isEmpty():

```
class QueueNode
{
 public:
 int val;
 QueueNode *next;
 QueueNode(int data)
  {val = data;
  next = nullptr;
  }};
```

Data Structures

CSL202

Quiz Set-D

1. Design and implement a **stack** using **queues**. The stack should support the following operations:

   push(x), pop(), top(), isEmpty()

   Input: push(10),push(20),push(30),pop(),top()

   Output: 30,20

2. Design and implement a **queue** using a **circular array**. The queue should support the following operations:

Input: enqueue(10),enqueue(20),

dequeue(),enqueue(40),dequeue()

Output: 10,20

2. Design and implement a **stack** using **queues**. The stack should support the following operations:

   push(x), pop(), top(), isEmpty()

   Input: push(10),push(20),push(30),pop(),top()

   Output: 30,20

3. There are `n` people in a queue to buy tickets. The `i`-th person wants `tickets[i]` tickets. Each ticket takes 1 second to buy. A person buys one ticket at a time, then goes to the back of the line if they still need more. Anyone finished buying leaves the line. Find how long it takes for the person at position `k` (0-indexed) to buy all their tickets.

   Input: tickets = [2,3,2], k = 2

   Output: 6

   Function: int timeRequiredToBuy(vector<int>& tickets, int k)

   {your code}

enqueue(x),dequeue(),peek(),isEmpty().

Input: enqueue(10),enqueue(20),dequeue(),enqueue(40),dequeue()

Output: 10,20

3. Aman and Akshat have a pile of N stones, each with an integer on it. They take turns making moves with Aman going first:

   Aman's turn: move a stone from the top to the bottom once, then remove the top stone.

   Akshat's turn: move a stone from the top to the bottom twice, then remove the top stone.

   They alternate turns until only one stone remains.

   Print 2 space-separated integers representing the person making the last move (1 for Aman and 0 for Akshat) and the number written on the stone remaining.

   Input 1 : stones = [-5,0,5]

   Output 1: 0,-5

   Input 2: stones = [-1 -3 2 4]

   Output 2: 1,2

   Function: void playStoneGame(vector<int>& stones)

   {*// Your code here*}